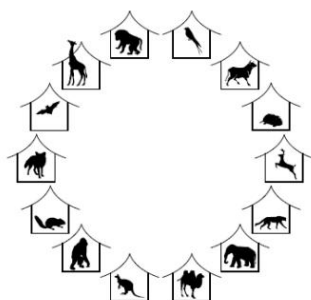


## 【APIO2007】动物园 1627

## Description

新建的圆形动物园是亚太地区的骄傲。圆形动物园坐落于太平洋的一个小岛上，它包含一大圈围栏，每个围栏里有一种富有异国风情的动物。如下图所示：



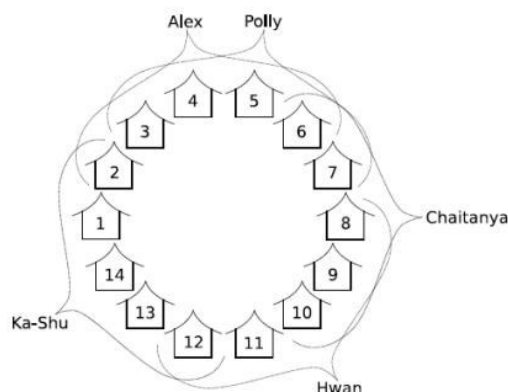
你是动物园的公关主管。你要做的是，让每个参观动物园的游客都尽可能高兴。今天有一群小朋友来到动物园参观，你希望能让他们在动物园度过一段美好的时光。但这并不是一件容易的事——有些小朋友喜欢某些动物，而有些小朋友则害怕某些动物。例如，Alex 喜欢可爱的猴子和考拉，而害怕张牙舞爪的狮子。而 Polly 会因狮子有美丽的鬃毛而喜欢它，但害怕有臭味的考拉。

你可以选择将一些动物从围栏中移走以使得小朋友不会害怕。但你移走的动物也不能太多，否则留给小朋友们观赏的动物就所剩无几了。

每个小朋友站在大围栏圈的外面，可以看到连续的 5 个围栏。你得到了所有小朋友喜欢和害怕的动物信息。当出现下面两种情形之一时，小朋友就会高兴：

至少有一个他害怕的动物（从视线可见的范围内）被移走，或  
至少有一个他喜欢的动物没有被（从视线可见的范围内）移走。

例如，考虑下图中的小朋友和动物：



| 小朋友       | 可见的围栏        | 害怕的动物       | 喜欢的动物  |
|-----------|--------------|-------------|--------|
| Alex      | 2,3,4,5,6    | 围栏 4        | 围栏 2,6 |
| Polly     | 3,4,5,6,7    | 围栏 6        | 围栏 4   |
| Chaitanya | 6,7,8,9,10   | 围栏 9        | 围栏 6,8 |
| Hwan      | 8,9,10,11,12 | 围栏 9        | 围栏 12  |
| Ka-Shu    | 12,13,14,1,2 | 围栏 12,13, 2 | -      |

假如你将围栏 4 和 12 的动物移走。Alex 和 Ka-Shu 将很高兴，因为至少有一个他们害怕的动物被移走了。这也会使 Chaitanya 高兴，因为他喜欢的围栏 6 和 8 中的动物都保留了。但是，Polly 和 Hwan 将不高兴，因为他们看不到任何他们喜欢的动物，而他们害怕的动物都还在。这种安排方式使得三个小朋友高兴。

现在换一种方法，如果你将围栏 4 和 6 中的动物移走，Alex 和 Polly 将很高兴，因为他们害怕的动物被移走了。Chaitanya 也会高兴，因为虽然他喜欢的动物 6 被移走了，他仍可以看到围栏 8 里面他喜欢的动物。同样的 Hwan 也会因可以看到自己喜欢的围栏 12 里的动物而高兴。唯一不高兴的只有 Ka-Shu。

如果你只移走围栏 13 中的动物，Ka-Shu 将高兴，因为有一个他害怕的动物被移走。Alex, Polly, Chaitanya 和 Hwan 也会高兴，因为他们都可以看到至少一个他们喜欢的动物。所以有 5 个小朋友会高兴。这种方法使得最多的小朋友高兴。

**Input**

输入的第一行包含两个整数  $N, C$ ，用空格分隔。 $N$  是围栏数 ( $10 \leq N \leq 10\,000$ )， $C$  是小朋友的个数 ( $1 \leq C \leq 50\,000$ )。围栏按照顺时针的方向编号为  $1, 2, 3, \dots, N$ 。

接下来的  $C$  行，每行描述一个小朋友的信息，以下面的形式给出：

$E \ F \ L \ X1 \ X2 \ \dots \ XF \ Y1 \ Y2 \ \dots \ YL$

其中：

$E$  表示这个小朋友可以看到的第一个围栏的编号 ( $1 \leq E \leq N$ )，换句话说，该小朋友可以看到的围栏为  $E, E+1, E+2, E+3, E+4$ 。注意，如果编号超过  $N$  将继续从  $1$  开始算。如：当  $N=14, E=13$  时，这个小朋友可以看到的围栏为  $13, 14, 1, 2$  和  $3$ 。

$F$  表示该小朋友害怕的动物数。 $L$  表示该小朋友喜欢的动物数。

围栏  $X1, X2, \dots, XF$  中包含该小朋友害怕的动物。

围栏  $Y1, Y2, \dots, YL$  中包含该小朋友喜欢的动物。

$X1, X2, \dots, XF, Y1, Y2, \dots, YL$  是两两不同的整数，而且所表示的围栏都是该小朋友可以看到的。

小朋友已经按照他们可以看到的第一个围栏的编号从小到大的顺序排好了(这样最小的  $E$  对应的小朋友排在第一个，最大的  $E$  对应的小朋友排在最后一个)。注意可能有多于一个小朋友对应的  $E$  是相同的。

**Output:**

仅输出一个数，表示最多可以让多少个小朋友高兴。

**【样例输入 1】**

```
14 5
2 1 2 4 2 6
3 1 1 6 4
6 1 2 9 6 8
8 1 1 9 12
12 3 0 12 13 2
```

**【样例输出 1】**

5

**【样例输入 2】**

```
12 7
1 1 1 1 5
5 1 1 5 7
5 0 3 5 7 9
7 1 1 7 9
9 1 1 9 11
9 3 0 9 11 1
11 1 1 11 1
```

**【样例输出 2】**

6

**【说明】**

上面的样例 1 给出了前面描述的示例情况。它使得所有小朋友 ( $C=5$ ) 高兴。样例 2 给出了这样的情形，要使得所有小朋友 ( $C=7$ ) 都高兴是不可能的。

**【分析】** 压缩动态规划

a. 学会位运算之后，这道题思路便十分清晰了

1) 枚举前 4 个动物的状态

2) 递推  $f[i, k]$  表示动规到以  $i$  为视野结束标志的小朋友为止，后 4 个窗口状态为  $k$  的时候，最多的高兴的小朋友

$f[i, k] := f[i-1, k \text{ and not } (1 \ll 4)] + Val$  (阶段  $i$  新增的高兴小朋友)

b. 细节注意

1) 小朋友的高兴和不高兴的表最好用 1 个 5 位二进制数存，这样判断的时候一个 check 函数就可以了  
 $exit((kid[j].x \text{ and not } k \gg 0) \text{ or } (kid[j].y \text{ and } k \gg 0));$

2) 注意破坏后 当  $i=1$  时， $i-1$  应该算  $n$

3) 思路一定要清晰，就能在百行内打完。

- 4) 其实压缩 dp 并不难
- 5) 注意结构化程序设计
- 6) 注意数据范围
- 7) 理解<<和>>
- 8) 深刻默写 zoo.pas
- 9) 调试方法: Procedure Bin + Debug 法, GDB break 法★
- 10) Believe yourself

```

Type kids=record B,L,F,X,Y :longint end;
  child=record S,B,T:longint end;
Var f:array [0..10000, 0..1<<4] of longint;
  kid:array [0..50000] of kids;
  c:array [0..10000] of child; r:longint;
  i,j,k,n,m,v,s,t,o:longint;
  ans,val,z:longint;
Procedure max(var a:longint; b:longint);
begin if b>a then a:=b;end;
Function check(j:longint):boolean;
begin
  exit((kid[j].x and not k<>0) or (kid[j].y and k<>0));
end;
Function Viable(i:longint):boolean;
begin
  if (i>4) and (i<9) then
    begin
      t:=k;
      for j:= 10-i to 5 do t:= t and not (1<<(j-1));
      if t<>s>>(i-5) then exit(false);
    end
  else if i<=4 then
    begin
      t:=s;
      for j:= i+1 to 4 do t:= t and not (1<<(j-1));
      if k>>(5-i)<>t then exit(false);
    end;
  exit(true);
end;
Procedure SDP(i:longint);
begin
  if c[v].b=i then
    begin
      for k:= 1<<5-1 downto 0 do
        begin
          if not viable(i) then continue;
          if i=1 then o:=f[n ,k and not (1<<4)]
            else o:=f[i-1,k and not (1<<4)]; val:=0;
          for j:= c[v].s to c[v].t do if Check(j) then inc(val);
          max(f[i,k>>1],val+o);
          max(ans,f[i,k>>1]);
        end;
      inc(v);
    end
end

```

```

else
  for k:= 1<<5-1 downto 0 do
    begin
      if not viable(i) then continue;
      if i<>1 then max(f[i,k>>1],f[i-1,k and not (1<<4)])
        else max(f[i,k>>1],f[n, k and not (1<<4)]);
    end;
  end;
end;
Begin
  readln(n,m);
  for i:= 1 to m do with kid[i] do
    begin
      read(b,f,l);
      for j:= 1 to f do
        begin
          read(z);
          if (b>=n-3) and (z<=4) then x:=x or (1<<(z+n-b))
            else x:=x or (1<<(z-b));
        end;
      for j:= 1 to l do
        begin
          read(z);
          if (b>=n-3) and (z<=4) then y:=y or (1<<(z+n-b))
            else y:=y or (1<<(z-b));
        end;
      inc(b,4); if b>n then dec(b,n);
      if b<>c[r].b then
        begin
          inc(r);
          c[r].b:=b;
          c[r].s:=i;
          c[r].t:=i;
        end
      else inc(c[r].t);
    end;
  end;
  for s:= 1<<4-1 downto 0 do
    begin
      fillchar(f,sizeof(f),0); v:=1;
      for i:= 5 to n do SDP(i);
      for i:= 1 to 4 do SDP(i);
    end;
  writeln(ans);
END.

```